

Professores:

Celso Giusti

Daniel Manoel Filho

Marlon Palata Fanger Rodrigues

Protocolos IoT e MQTT na Prática



POR QUE PRECISAMOS DE PROTOCOLOS?

Por que precisamos de protocolos de comunicação?

Sensores geram dados. Mas dados presos no dispositivo não servem para nada.

O protocolo é o idioma que os dispositivos usam para se falar.

Assim como humanos precisam falar o mesmo idioma para se entender, dispositivos IoT precisam de um protocolo em comum para trocar informações.

O que um protocolo define:

- Como os dados são formatados
- Como a conexão é iniciada e encerrada
- O que acontece se uma mensagem se perder
- Quem pode falar com quem

Dica: Pense no protocolo como as regras de um jogo. Todos os jogadores precisam seguir as mesmas regras para que o jogo funcione.

OS 6 PROTOCOLOS IoT

Os principais protocolos do ecossistema IoT

Não existe protocolo perfeito. Cada um foi criado para resolver um problema específico.

Protocolo	Criado para	Alcance	Consumo	Velocidade
MQTT	Envio de dados leves via rede	Rede local / Internet	Muito baixo	Médio
HTTP	Comunicação web clássica	Internet	Alto	Alto
BLE	Curta distância, baixo consumo	~10–100m	Muito baixo	Baixo
Zigbee	Redes mesh de sensores	~10–75m por nó	Muito baixo	Baixo
LoRaWAN	Longas distâncias, pouco dado	Até 15 km	Extremamente baixo	Muito baixo
NB-IoT / LTE-M	Celular para IoT	Nacional	Baixo–médio	Médi

Importante: Nesta aula vamos usar o **MQTT**. Os demais são abordados teoricamente para você saber quando e por que escolher cada um.

MQTT – Message Queuing Telemetry Transport

Criado pela IBM em 1999 para monitorar oleodutos no deserto via satélite.

O problema: link de satélite era caro e instável. Precisava de um protocolo leve, confiável mesmo com conexão ruim.

A solução: MQTT — mensagens pequenas, reconexão automática, baixíssimo consumo.

Hoje é o protocolo padrão da indústria IoT.

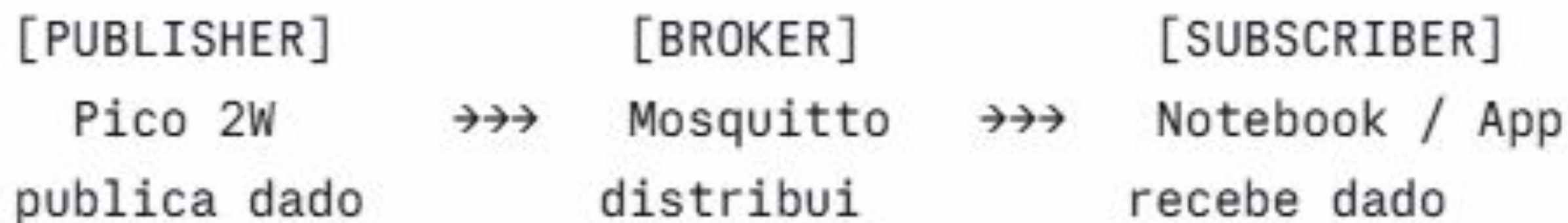
Características:

- Roda sobre TCP/IP
- Porta padrão: **1883** (sem criptografia) / **8883** (com TLS)
- Modelo **Publish/Subscribe** (Publicar/Assinar)
- Precisa de um intermediário: o **Broker**
- QoS (Quality of Service): 0, 1 ou 2

Lembre-se: O Pico 2W vai usar MQTT para enviar dados dos sensores. Por isso estudamos ele primeiro e em mais profundidade.

MQTT: PUBLISH / SUBSCRIBE

Diferente do HTTP (que é pedido → resposta), o MQTT funciona como uma rádio FM.



As peças:

- **Publisher** → quem envia a mensagem (ex: o Pico com o DHT22)
- **Broker** → o intermediário que recebe e distribui (ex: Mosquitto)
- **Subscriber** → quem recebe a mensagem (ex: o seu navegador)
- **Tópico** → o endereço da mensagem (ex: **senai/sala1/temperatura**)

Atenção: Publisher e Subscriber **nunca se falam diretamente**. Tudo passa pelo Broker. É por isso que um único broker pode conectar 34 dispositivos ao mesmo tempo.

MQTT: TÓPICOS

MQTT: A estrutura dos tópicos

O tópico é como um endereço de pasta. Usa / para separar os níveis.

Exemplos de tópicos:

```
senai/sala1/temperatura
senai/sala1/umidade
senai/sala2/presenca
casa/quarto/led/estado
empresa/fabrica/maquina3/rpm
```

Wildcards (curingas) no Subscribe:

```
senai/sala1/#      → recebe TUDO de sala1
senai/+/temperatura → recebe temperatura de
QUALQUER sala
```

Dica: Defina uma convenção de nomenclatura antes de começar o projeto. Tópicos bagunçados são difíceis de manter. Use sempre letras minúsculas, sem espaços, separados por /.

Importante: Tópicos começando com \$ são reservados pelo broker (ex: \$SYS/broker/clients/connected).

MQTT: QoS

MQTT: Níveis de Qualidade de Serviço (QoS)

O QoS define o quanto o MQTT garante a entrega da mensagem.

Nível	Nome	Garantia	Uso típico
QoS 0	At most once	Manda e esquece. Pode perder.	Temperatura em tempo real
QoS 1	At least once	Garante entrega, pode duplicar	Alarmes, notificações
QoS 2	Exactly once	Garante entrega única, mais lento	Transações, pagamentos

Lembre-se: Maior QoS = mais tráfego de rede = mais consumo de bateria. Para IoT em geral, **QoS 0 ou 1** são os mais usados.

Nesta aula usaremos QoS 0 — simples e suficiente para aprender o fluxo completo.

HTTP

HTTP — Hypertext Transfer Protocol

O protocolo que você usa toda vez que abre um site.

Modelo: Requisição → Resposta (Request/Response)

Cliente pede → Servidor responde → Conexão encerra

Por que não é ideal para IoT?

- Cada mensagem carrega muito "peso" (headers, verbos, status codes)
- O dispositivo precisa **perguntar** se tem dado novo (polling) — gasta bateria
- Não há notificação automática — o servidor não "avisa" o cliente

Quando usar HTTP em IoT:

- Configuração remota de dispositivos
- Integração com APIs REST (ex: enviar dado para um servidor web)
- Quando segurança e compatibilidade com web são prioridade

Importante: O Pico 2W consegue fazer requisições HTTP. Mas para dados contínuos de sensores, o MQTT é muito mais eficiente.

Bluetooth Clássico: para transferência de arquivos, áudio. Alto consumo.

BLE (Bluetooth Low Energy): criado para dispositivos que funcionam anos com uma pilha.

Características do BLE:

- Alcance: 10 a 100 metros
- Consumo: extremamente baixo (pode durar anos com pilha de botão)
- Velocidade: baixa (~1 Mbps)
- Conexão: ponto a ponto ou broadcast (beacon)
- Sem necessidade de infraestrutura de rede

Casos de uso típicos:

- Smartwatches e pulseiras fitness
- Rastreadores (AirTag, Tile)
- Fechaduras inteligentes
- Sensores médicos

Atenção: BLE não usa WiFi nem internet. A comunicação é direta entre dois dispositivos próximos. Para enviar dado à nuvem, precisaria de um gateway intermediário.

ZIGBEE

Criado para redes de sensores onde dezenas ou centenas de dispositivos precisam se comunicar com baixíssimo consumo.

O diferencial: rede Mesh

[Sensor A] → [Sensor B] → [Sensor C] → [Gateway] → Internet

↑ repassa a mensagem do A

Cada dispositivo pode **repassar mensagens de outros**. Se um nó falhar, a rede encontra outro caminho.

Características:

- Alcance: 10–75m por nó (mas a rede pode cobrir áreas enormes via mesh)
- Consumo: muito baixo
- Velocidade: baixa (250 kbps)
- Capacidade: até 65.000 dispositivos na mesma rede

Casos de uso:

- Automação residencial (Philips Hue, IKEA Tradfri)
- Automação industrial com muitos sensores
- Redes de medidores elétricos inteligentes

LoRaWAN

Long Range Wide Area Network

O protocolo para lugares onde não tem WiFi, não tem celular, mas precisa de monitoramento.

Características:

- Alcance: 2 a 15 km em área aberta
- Consumo: extremamente baixo (bateria dura anos)
- Velocidade: muito baixa (< 50 kbps) — só dados pequenos
- Infraestrutura: precisa de gateway LoRa + servidor de rede

Casos de uso:

- Rastreamento de gado em fazendas
- Monitoramento de enchentes em rios remotos
- Medição de qualidade do ar em cidades
- Rastreamento de containers portuários
- Irrigação inteligente em lavouras

Importante: LoRaWAN não é internet. É uma rede dedicada. Os dados chegam a um gateway que então os envia à nuvem via internet.

Lembre-se: A sigla "Lora" vem de "Long Range" — o foco é o alcance, não a velocidade.

NB-IoT e LTE-M

IoT via rede celular

A ideia: usar a infraestrutura de celular (torres 4G/5G) para conectar dispositivos IoT.

	NB-IoT	LTE-M
Velocidade	Muito baixa (~250 kbps)	Baixa (~1 Mbps)
Consumo	Extremamente baixo	Baixo
Mobilidade	Estático (sem roaming)	Suporta movimento
Latência	Alta (segundos)	Baixa (milissegundos)
Uso típico	Medidores de água/luz	Rastreadores veiculares

Vantagem principal: usa a rede celular existente. Não precisa instalar gateway ou infraestrutura própria.

Desvantagem: custo de chip SIM + plano de dados por dispositivo.

Dica: Os medidores inteligentes de água e gás que as concessionárias estão instalando nas casas usam NB-IoT. É por isso que o técnico não precisa mais ir até sua casa para fazer a leitura.

Comparativo Final Dos Protocolos

Quando usar cada protocolo?

Cenário	Protocolo ideal	Por quê
Enviar temp/umidade para a nuvem via WiFi	MQTT	Leve, eficiente, pub/sub
Integrar com uma API REST existente	HTTP	Compatibilidade universal
Smartwatch enviando passos para o celular	BLE	Curta distância, baixíssimo consumo
200 lâmpadas inteligentes na mesma casa	Zigbee	Rede mesh, muitos dispositivos
Monitorar rio a 10 km sem infraestrutura	LoRaWAN	Longo alcance, bateria anos
Rastrear caminhão em movimento no Brasil	LTE-M	Cobertura celular + mobilidade

Importante: Na prática real, sistemas IoT complexos usam **múltiplos protocolos**. O sensor usa BLE → gateway usa MQTT → dashboard usa HTTP. Cada camada escolhe o protocolo certo para o seu problema.

CHEGOU A HORA DA PRÁTICA

Teoria vista. Agora vamos ver o MQTT funcionando de verdade.

O que vamos fazer hoje:

- **PASSO 1** → Baixar o Mosquitto (broker local)
- **PASSO 2** → Configurar e rodar o broker
- **PASSO 3** → Testar com pub/sub no terminal
- **PASSO 4** → Conectar o Pico 2W no WiFi
- **PASSO 5** → Publicar "Hello MQTT" pelo Pico
- **PASSO 6** → Ver a mensagem chegar no terminal
- **PASSO 7** → BÔNUS: ver chegando no HiveMQ (nuvem)

Lembre-se: O Pico está conectado via USB para energia e upload de código. A comunicação MQTT acontece via WiFi — são coisas separadas.

O MOSQUITTO – BROKER

O que é: broker MQTT open source, mantido pela Eclipse Foundation. Padrão da indústria.

Antes da prática: duas formas de usar o broker

Hoje vamos praticar das **duas formas**. Começamos pela mais simples.

	Opção A — HiveMQ (nuvem)	Opção B — Mosquitto (local)
Instalação	Nenhuma	Instalar o .exe (admin)
Broker roda onde	Servidor em Frankfurt 🌐	No seu próprio notebook 💻
Pico conecta em	broker.hivemq.com	IP do seu notebook
Ver as mensagens	No navegador (site HiveMQ)	No terminal (mosquitto_sub)
Depende de internet	✅ Sim	❌ Não
Risco de firewall	❌ Nenhum	⚠️ Possível

Sequência de hoje:

- 1. Primeiro:** Opção A — HiveMQ direto. Pico publica, vocês veem no navegador.
- 2. Depois:** Opção B — Mosquitto local. Configuram o próprio broker.

Importante: O código do Pico é **quase idêntico** nas duas opções. Só muda o endereço do broker em uma linha do **config.py**.

OPÇÃO A: HIVEMQ

Opção A — Como funciona

Pico 2W

↓ WiFi (rede IoT)

broker.hivemq.com

← broker gratuito na nuvem

↓

Navegador de vocês (site HiveMQ)

O Pico vai:

- Conectar na rede **IoT**
- Publicar dados do DHT22 a cada 3 segundos
- Tópico: **senai/seunome/dht22**

Atenção: O broker é **público**. Use sempre seu nome no tópico para não misturar com outros usuários do mundo.

OPÇÃO A: SUBSCRIBER NO NAVEGADOR

Opção A — Abrindo o HiveMQ

Acesse: **hivemq.com/demos/websocket-client**

Host: `broker.hivemq.com`

Port: `8000`

→ Clique **Connect**

Topic: `senai/seunome/dht22`

→ Clique **Subscribe**

Dica: A porta aqui é **8000** (WebSocket). O Pico usa **1883** (TCP direto). O broker aceita os dois — são portas diferentes para o mesmo serviço.

OPÇÃO A: CONFIG.PY

Opção A — Configurando o Pico

```
# config.py
WIFI_SSID = "IoT"
WIFI_PASS = "senai1234"
BROKER_IP = "broker.hivemq.com" # ← HiveMQ
BROKER_PORT = 1883
CLIENT_ID = "pico_seunome" # ← use seu nome!
TOPIC_PUB = "senai/seunome/dht22" # ← use seu nome!
```

Importante: Cada aluno usa um **CLIENT_ID** diferente. IDs iguais fazem o broker desconectar o dispositivo anterior.

Lembre-se: Salvar no Pico → Arquivo → Salvar no dispositivo MicroPython

OPÇÃO B: O QUE É O MOSQUITTO

Opção B — Mosquitto: broker no seu notebook

O que é: Broker MQTT open source, padrão da indústria. Roda localmente.

Por que usar:

- Sem depender de internet
- Você controla o broker
- Experiência completa de infraestrutura IoT

Download:

mosquitto.org/download → Windows → .exe

Após instalado, vira um **serviço do Windows** — roda em background automaticamente.

Lembre-se: Instalado antes da aula pelo professor (precisa de admin).

OPÇÃO B: EDITANDO O MOSQUITTO.CONF

Opção B — Liberando conexões externas

Por padrão, o Mosquitto só aceita conexões do próprio computador. Precisamos mudar isso.

Arquivo:

C:\Program Files\mosquitto\mosquitto.conf

(Bloco de Notas como administrador)

Adicionar no final:

listener 1883

allow_anonymous true

- **listener 1883** → aceita conexões de qualquer IP
- **allow_anonymous true** → sem senha (só para testes!)

Atenção: Só funciona após reiniciar o serviço — próximo slide.

OPÇÃO B: REINICIANDO O SERVIÇO

Opção B — Aplicando a nova configuração

Após editar o **mosquitto.conf**, reinicie o serviço.

Via terminal (como administrador):

```
net stop mosquitto
```

```
net start mosquitto
```

Via interface:

Painel de Controle → Ferramentas Administrativas → Serviços → Mosquitto → Reiniciar

Confirmar que está rodando:

```
mosquitto -v
```

Lembre-se: O serviço roda em background — sem janela aberta. Isso é normal.

OPÇÃO B: DESCOBRINDO O IP

Opção B — Qual IP colocar no Pico?

No terminal:

```
ipconfig
```

Procure:

Adaptador de Rede sem Fio Wi-Fi:

Endereço IPv4 . . . : 192.168.X.XXX ← esse aqui

Atenção: Não use localhost no Pico. Localhost é o próprio dispositivo — para o Pico, localhost seria ele mesmo. O broker está no notebook, então use o IP do notebook.

Lembre-se: Na rede da escola os IPs são fixos. Anote o seu.

OPÇÃO B: TESTANDO O BROKER

Opção B — Teste antes de conectar o Pico

Terminal 1 — deixa aberto:

```
mosquitto_sub -h localhost -t "senai/seunome/dht22"
```

Terminal 2 — teste manual:

```
mosquitto_pub -h localhost -t "senai/seunome/dht22" -m "teste ok"
```

Se aparecer teste ok no Terminal 1 → broker OK ✓

Só depois conectamos o Pico.

Dica: Esse teste isola o problema. Se falhar aqui, o problema é no broker — não no Pico. Facilita o diagnóstico.

OPÇÃO B: TROCANDO O BROKER NO CÓDIGO

Opção B — Uma linha muda tudo

No **config.py** do Pico, mude **só essa linha**:

Antes (HiveMQ):

```
BROKER_IP = "broker.hivemq.com"
```

Depois (Mosquitto local):

```
BROKER_IP = "192.168.X.XXX" # IP do seu notebook
```

Todo o resto permanece igual: wifi_connect.py ✓ | main.py ✓ | CLIENT_ID ✓ | TOPIC_PUB ✓

Salve e rode o **main.py**. Observe as mensagens chegando no Terminal 1.

Lembre-se: Trocar de infraestrutura sem tocar na lógica. Esse é o valor de separar configuração do código.

Resumo

- ✓ Protocolo = "idioma" que dispositivos usam para se comunicar.
- ✓ MQTT = protocolo leve, pub/sub, ideal para IoT. Porta 1883.
- ✓ HTTP = requisição/resposta, pesado para IoT contínuo.
- ✓ BLE = bluetooth de baixo consumo, curta distância (~100m).
- ✓ Zigbee = rede mesh de sensores, até 65k dispositivos.
- ✓ LoRaWAN = longo alcance (até 15km), bateria de anos, dados pequenos.
- ✓ NB-IoT / LTE-M = IoT via rede celular, cobertura nacional.
- ✓ Broker = intermediário MQTT que distribui as mensagens.
- ✓ Publisher = quem envia. Subscriber = quem recebe. Tópico = endereço.
- ✓ Mosquitto = broker open source, portátil (ZIP), sem instalação.
- ✓ config.py = separar credenciais do código principal (boa prática).
- ✓ umqtt.simple = biblioteca MQTT nativa do MicroPython do Pico 2W.
- ✓ Trocar broker local ↔ nuvem = mudar apenas o endereço em config.py.

MQTT

1

Exercício 1

Objetivo: Pub/Sub no terminal

Com o Mosquitto rodando, abra dois terminais e faça uma conversa com você mesmo: publique 5 mensagens diferentes no tópico **senai/seuname/chat** e confirme que todas aparecem no subscriber.

MQTT

2

Exercício 2

Objetivo: Descobrir o IP

Abra o terminal e execute **ipconfig**. Identifique e anote o IP do seu notebook na rede IoT. Explique por escrito (2-3 linhas) por que o Pico não pode usar **localhost** para se conectar ao seu broker.

MQTT

3

Exercício 3

Objetivo: Modificando o Pico

Altere o **main.py** para que o Pico publique no tópico **senai/seuname/contador**, enviando apenas o número da mensagem (1, 2, 3...) em vez do texto completo. Teste e confirme no terminal.

MQTT

4

Exercício 4

Objetivo: Tópicos hierárquicos

Crie um segundo subscriber que escuta **senai/#** (todos os tópicos de senai). Publique em 3 tópicos diferentes e confirme que o subscriber com **#** recebe todos. Explique por escrito o que o **#** significa.

MQTT

5

Exercício 5

Objetivo: Reflexão de protocolo

Escolha um dos cenários abaixo e justifique qual protocolo você usaria e por quê (mínimo 5 linhas):

- a) Monitorar temperatura de servidores em um datacenter
- b) Rastrear encomendas em caminhões pelo Brasil
- c) Controlar luzes de um apartamento pelo celular
- d) Medir nível de chuva em estações meteorológicas no interior do Amazonas

ATIVIDADE

MQTT

6

Exercício 6

Objetivo: BÔNUS: HiveMQ

Repita o Exercício 3, mas apontando o broker para **broker.hivemq.com**. Faça um colega assinar o seu tópico no HiveMQ WebSocket Client e tire um print como comprovante.



SENAI. **IOT – Internet das Coisas**. Brasília: SENAI/DN, 2012. p. 69–130.

GIUSTI, Celso Rodrigo. **Técnicas de Codificação e Design para Sistemas Back-End**. São Paulo: SENAI, 2025.

OASIS. **MQTT Version 3.1.1**. OASIS Standard, 2014. Disponível em: <https://docs.oasis-open.org/mqtt/mqtt/v3.1.1/mqtt-v3.1.1.html>. Acesso em: abr. 2026.

ECLIPSE FOUNDATION. **Eclipse Mosquitto: An open source MQTT broker**. Disponível em: <https://mosquitto.org>. Acesso em: abr. 2026.

HIVEMQ. **MQTT Essentials**. Disponível em: <https://www.hivemq.com/mqtt-essentials>. Acesso em: abr. 2026.

RASPBERRY PI FOUNDATION. **Raspberry Pi Pico 2W Datasheet**. Disponível em: <https://www.raspberrypi.com/documentation/microcontrollers/pico-series.html>. Acesso em: abr. 2026.

MICROPYTHON. **umqtt.simple – MQTT client library**. Disponível em: <https://github.com/micropython/micropython-lib/tree/master/micropython/umqtt.simple>. Acesso em: abr. 2026.



FOTO DA ESCOLA

ESCOLA SENAI ÍTALO BOLOGNA

Av. Goiás, 139

Telefone

(11) 2396-1999

Instagram

@senai.itu

Facebook

/senaisp.itu

Site

<https://sp.senai.br/unidade/itu/>